# ILForge — the languages

Seventeen implementations: thirteen compilers that emit **pure .NET IL**, three interpreters, and a calculator. This page is the map — what each one *is*, what it *does*, how to *run* it, and what it deliberately leaves out. Each entry links to a full reference whose tutorial examples were all compiled and run, with real output.

Everything below is a **subset** of its language, honestly bounded: the goal is a faithful, working core of each language rather than a conformance-complete implementation. Every tool accepts `-h`, or prints usage when run with no arguments.

## At a glance

| Language | Tool | Source | → `.exe` | → `.dll` (C#/VB interop) | Reference |
|---|---|---|---|---|---|
| C | `cc` | `.c` | ✓ | ✓ | lex-yacc.md |
| Pascal | `pascal` | `.pas` | ✓ | — | pascal.md |
| Modula-2 / Oberon-2 | `oberon` | `.mod` | ✓ | — | oberon.md |
| Tiny C++ | `tcpp` | `.cpp` | ✓ | — | tcpp.md |
| QBasic | `qbasic` | `.bas` | ✓ | — | qbasic.md |
| Forth | `forth` | `.fth` | ✓ | — | forth.md |
| Fortran 90 | `fortran` | `.f90` | ✓ | ✓ | fortran.md |
| COBOL | `cobol` | `.cob` | ✓ | ✓ | cobol.md |
| Ada | `ada` | `.adb` | ✓ | ✓ | ada.md |
| Smalltalk | `smalltalk` | `.st` | ✓ | ✓ | smalltalk.md |
| Lua | `lua` | `.lua` | ✓ | ✓ | lua.md |
| AWK | `awk` | `.awk` | ✓ | ✓ | awk.md |
| Coil | `coilfe` | `.coil` | ✓ | ✓ | coil.md |
| Logo | `logo` | `.logo` | interpreted | — | logo.md |
| Lisp | `lisp` | `.lisp` | interpreted | — | lisp.md |
| Prolog | `prolog` | `.pl` | interpreted | — | prolog.md |
| bc | `bc` | — | REPL | — | bc.md |

Common shape for the compilers:

```
<tool> <source> [-o <output>]          # produce output.exe (+ output.dll)
<tool> <source> -o lib.dll --dll       # produce a library for C#/VB.NET  (where supported)
```

`cc` additionally takes `-I <dir>` (include search), `-L <dir>` / `-l <name>` (library search and staging), `--icon <file>` / `--noicon`, and honours `INCLUDE` and `LIB` from the environment — the ILForge Developer Command Prompt sets those, as a Visual Studio prompt does.

---

## The compilers

### C — `cc`

The foundation: a C89-plus compiler that lowers C to IL directly. Everything else in ILForge is built with it, including `lex`, `yacc`, and the other compilers.

- **Implemented:** the C89 core — `int` / `char` / `double` / `long`, pointers, arrays, `struct` / `union`, `typedef`, `enum`, functions, all the operators, `static` locals, the full statement set, a real preprocessor (`#define` object- and function-like, `#include`, `#ifdef` / `#if` / `#elif` / `#else`, line continuations), and a large built-in libc (`printf` family, `str*`, `mem*`, `malloc` / `free`, `fopen` / `fgets` / `fputs`, `atoi` / `atof` / `strtod`, `math`).
- **Memory model:** one flat byte arena; a C pointer is an `int` offset into it. That is what makes arbitrary C pointer arithmetic expressible in verifiable IL.
- **Interop:** `--dll` exposes each function as a `public static` method on `CProgram`, so C# calls it with ordinary .NET signatures.
- **Not implemented:** `float` as a distinct type (all floating point is `double`), bitfields, `goto` into blocks, variadic *user* functions (libc varargs work), `setjmp` / `longjmp`, and the wider C99/C11 additions.
- **Function pointers** carry int-sized signatures: a function whose parameters or result are `double` or `long` cannot be called *through a pointer* (the indirect-call dispatcher passes everything as a 32-bit int). Direct calls to such functions are unaffected.

### Pascal — `pascal`

Turbo-Pascal-flavoured Pascal. See pascal.md for the verified feature list, tutorial and boundaries.

### Modula-2 / Oberon-2 — `oberon`

Wirth's successors to Pascal, sharing one front end. See oberon.md.

### Tiny C++ — `tcpp`

C with classes, and the polymorphism that follows from them.

- **Implemented:** C scalars, pointers, arrays, `struct`, and **classes** — a class is a struct with a hidden vtable pointer at offset 0, so a derived pointer is layout-compatible with its base. Single inheritance,

`virtual` methods and real dynamic dispatch, constructors, `new` / `delete`, member functions, C strings.
- **Not implemented:** templates, the STL, namespaces, operator overloading, multiple inheritance, exceptions, destructors at scope exit (RAII), `auto`/range-`for`, `std::string`. `#include` is unnecessary (libc is built in) and not processed.

### QBasic — `qbasic`

The Microsoft BASIC most people met first, including its graphics.

- **Implemented:** suffix-typed variables (`%` integer, `&` long, `!` single, `#` double, `$` string; unsuffixed numerics are double), arrays with constant bounds, `SUB` and `FUNCTION` with **by-reference** parameters, `IF` / `ELSEIF`, `FOR` / `NEXT` with `STEP`, `WHILE` / `WEND`, `DO` / `LOOP`, `SELECT CASE`, `GOTO`/labels, `PRINT` / `INPUT`, string functions, and a graphics word set (`SCREEN`, `PSET`, `LINE`, `CIRCLE`, `PAINT`) that renders in the `ilgfx` window — run those with `gfx <name>` from the shell. Emits `#line` info, so compiled BASIC is debuggable at source level.
- **Not implemented:** `GOSUB` / `RETURN`, `$INCLUDE`, `TYPE…END TYPE` records, `READ` / `DATA`, fixed-length strings, `PRINT USING`, distinct single vs double precision, event traps.

### Forth — `forth`

A stack language on a .NET stack: the data stack is a `Stack<object>`, so cells hold `int`, `double`, or `string` and the operators are polymorphic by runtime type.

- **Implemented:** the core word set (arithmetic, comparison, stack shuffling, `.` / `."`), colon definitions `: name … ;` compiled to real methods, `IF` / `ELSE` / `THEN`, `DO` / `LOOP`, `BEGIN` / `UNTIL` / `WHILE` / `REPEAT`, `VARIABLE` / `CONSTANT`, `@` / `!`, and string literals `S" …"`.
- **Not implemented:** cell-array words (`CREATE` / `ALLOT` / `CELLS`), return-stack words (`>R` / `R>`), `DOES>`/defining words, `IMMEDIATE`/compile-time metaprogramming, `ABORT`/exceptions, floating-point formatting words. `VARIABLE` gives one cell.

### Fortran 90 — `fortran`

Free-form Fortran with the numerics that make it Fortran.

- **Implemented:** `PROGRAM` / `SUBROUTINE` / `FUNCTION` (subroutine arguments **by reference**, function arguments by value), `INTEGER` / `REAL` / `LOGICAL` / `CHARACTER`, arrays, `IF` / `THEN` / `ELSE`, `DO` loops, `DO WHILE`, arithmetic and the intrinsic functions, `PRINT` / `WRITE`.
- **Interop:** each procedure becomes `CProgram.f_<name>`.
- **Full detail:** fortran.md.

### COBOL — `cobol`

The most complete of the set, and the one that behaves most like the original.

- **Implemented:** all four divisions; `PICTURE` parsing with edited fields; group and record hierarchies with `OCCURS` tables, `VALUE`, 88-level condition names, and `OF` / `IN` qualification; `MOVE` (including group moves and `CORRESPONDING`), arithmetic verbs with `ROUNDED`, `PERFORM` (`TIMES` / `UNTIL` / `VARYING` / `THRU`), `IF` / `ELSE`, class and sign conditions,

`STRING` / `UNSTRING` / `INSPECT`, `SET`, `INITIALIZE`, reference modification, intrinsic `FUNCTION`s, subprograms with `LINKAGE SECTION` + `CALL` / `GOBACK`, and **line-sequential file I/O** (`SELECT` / `ASSIGN`, `FD`, `OPEN` / `CLOSE` / `READ` / `WRITE` with `FILE STATUS`).
- **Not implemented:** `SEARCH`, `EVALUATE TRUE`, inline `AT END` on `READ`, `REDEFINES`, fixed-format source, indexed/relative files, `REWRITE`, `COPY`.

### Ada — `ada`

Strong typing, and the parameter modes that Ada is known for.

- **Implemented:** `Integer` / `Float` / `Boolean` / `Character` / `String`, enumerations, 1-based arrays, functions and procedures with `in` (by value) and `out` / `in out` (**by reference**) parameters, `if` / `elsif` / `else`, `case` / `when` / `others`, `loop` / `while` / `for` (including `reverse`) with `exit when`, `'Image`, `&` concatenation, `Ada.Text_IO`. Case-insensitive, as Ada is.
- **Not implemented:** packages with separate spec/body, generics, tasking, exceptions, `record` types, access types, `declare` blocks, 2-D arrays and array attributes.

### Smalltalk — `smalltalk`

Everything is an object; every operation is a message send, dispatched at run time.

- **Implemented:** boxed objects for all values, the classic **unary > binary > keyword** precedence (binary strictly left-to-right — `3 + 4 * 2` is 14), class definitions with instance variables and methods, `^` return, `self`, `Class new`, and the control-flow messages (`ifTrue:ifFalse:`, `whileTrue:`, `to:do:`, `timesRepeat:`) which are recognised and compiled inline.
- **Interop:** at the object-runtime level — C# creates instances and sends messages with selector strings.
- **Not implemented:** collection classes and their `do:` / `collect:` protocol, blocks as first-class values (they work only as inlined control-flow arguments), class-side methods, inheritance beyond one level, cascades, metaclasses.

### Lua — `lua`

Dynamically typed, built on the table.

- **Implemented:** nil/boolean/number/string/table/function; **tables** doing array and hash duty together (1-based, `#`, `t.k` sugar); first-class functions, lambda-lifted to real methods so they can be stored in tables and passed around; `:` method sugar with `self`; multiple assignment (`a, b = b, a`); numeric and generic (`pairs` / `ipairs`) `for`, `while`, `repeat` / `until`, `if` / `elseif`; `print` / `type` / `tostring` / `tonumber`.
- **Not implemented:** closures that capture an enclosing function's locals (named functions are global, so recursion works), multiple return values, metatables, varargs, `goto`, the integer/float distinction (all numbers are doubles).

### AWK — `awk`

Pattern-action text processing over standard input.

- **Implemented:** `BEGIN` / `END` and `pattern { action }` rules, fields ( `$0` / `$1` / `NF` / `NR` ) with `FS` / `OFS` / `ORS` , dynamic string-or-number values, associative arrays with `for (k in a)` and `delete` , regex patterns and `~` / `!~` , **string concatenation by juxtaposition**, all the operators, `if` / `while` / `do` / `for` (both forms), `next` / `exit` , user functions, and the built-ins `length` , `substr` , `index` , `split` , `sprintf` , `toupper` , `tolower` , plus the math set.
- **Not implemented:** `sub` / `gsub` / `match` , `getline` , multiple input files ( `FNR` , `FILENAME` ), output redirection, arrays as function arguments, regex alternation/grouping.

### Coil — `coilfe`

A deliberately tiny curly-brace language that sits roughly 1:1 with IL — the clearest window onto what the CLR actually executes. Uniquely, its back end is a C# `Reflection.Emit` assembler ( `coilasm` ) rather than `cc` .

- **Implemented:** five CLR primitive types, functions, `if` / `else` , `while` , locals/params, arithmetic and comparison, `print` / `println` .
- **Not implemented:** arrays/collections, `for` / `do` , input, user-defined types, global variables, calls to arbitrary .NET methods, wider numeric types, exceptions.

---

## The interpreters

### Logo — `logo`

Turtle graphics that render to a file: `logo prog.logo -png out.png` (also `-svg` and animated `-gif` ); no arguments gives a REPL.

- **Implemented:** numbers, words ( `"hello` ), lists ( `[a b c]` , also command blocks), the turtle words ( `FORWARD` / `BACK` / `RIGHT` / `LEFT` / `PENUP` / `PENDOWN` / `SETPENCOLOR` ), `REPEAT` with `:repcount` , `IF` / `IFELSE` , `TO … END` procedures with inputs and recursion, arithmetic, `PRINT` .
- **Not implemented:** arrays, full list processing ( `FIRST` / `BUTFIRST` / `FPUT` ), property lists, `RUN` , dynamic `THING` , most of a full Logo library.

### Lisp — `lisp`

A cons-cell Lisp with closures, and a metacircular evaluator written in itself.

- **Implemented:** numbers, interned symbols, strings, `()` , cons pairs, lambda closures; `define` / `lambda` / `if` / `cond` / `let` / `begin` / `quote` / `set!` , the list and arithmetic primitives, `apply` , `eval` , and a prelude written in Lisp.
- **Not implemented:** macros, tail-call optimization (deep non-tail recursion can overflow), vectors/hash tables, string manipulation beyond literals, `call/cc` , exceptions, the full numeric tower.

### Prolog — `prolog`

Real unification with a binding trail, SLD resolution, backtracking, and the cut.

- **Implemented:** atoms, numbers, variables, compound terms, lists ( `[a, b | T]` ); facts and rules, conjunction, the cut `!` , arithmetic via `is` , comparison, if-then-else `(C → T ; E)` , and query output.
- **Not implemented:** `assert` / `retract` , `findall` / `bagof` / `setof` , `op/3` , a real string type, DCGs, exceptions, full I/O.

### bc — `bc`

A scientific calculator: `bc "2+3*4"` , or a REPL with no arguments.

- **Implemented:** one type ( `double` ), full operator precedence, parentheses, named variables, `pi` and `e` , and the scientific function set.
- **Not implemented:** `define` functions, statements ( `if` / `while` / `for` ), arrays, arbitrary precision ( `scale` ), strings.

---

## Building your own

The same `lex` + `yacc` + `cc` pipeline that produced every compiler above is documented in **lex-yacc.md**, with two worked examples — a calculator and a small shell — under `examples/lexyacc/` .